

caller, and so on up the stack of calls. As we'll see, our main code will handle this exception.

Optimized Bank Class

The full `Bank` class code is available for download. In [Listing 4-10](#) I show some sample methods that demonstrate `try/except` techniques with calls to methods in the previously updated `Account` class.

File: BankOOP6_UsingExceptions/Bank.py (modified to work with previous Account.py)

```
# Bank that manages a dictionary of Account objects

from Account import *

class Bank():
    def __init__(self, hours, address, phone): ❶
        self.accountsDict = {}
        self.nextAccountNumber = 0
        self.hours = hours
        self.address = address
        self.phone = phone

    def askForValidAccountNumber(self): ❷
        accountNumber = input('What is your account number? ')
        try: ❸
            accountNumber = int(accountNumber)
        except ValueError:
            raise AbortTransaction('The account number must be an integer')
        if accountNumber not in self.accountsDict:
            raise AbortTransaction('There is no account ' + str(accountNumber))
        return accountNumber

    def getUsersAccount(self): ❹
        accountNumber = self.askForValidAccountNumber()
        oAccount = self.accountsDict[accountNumber]
        self.askForValidPassword(oAccount)
        return oAccount

    --- snipped additional methods ---
```